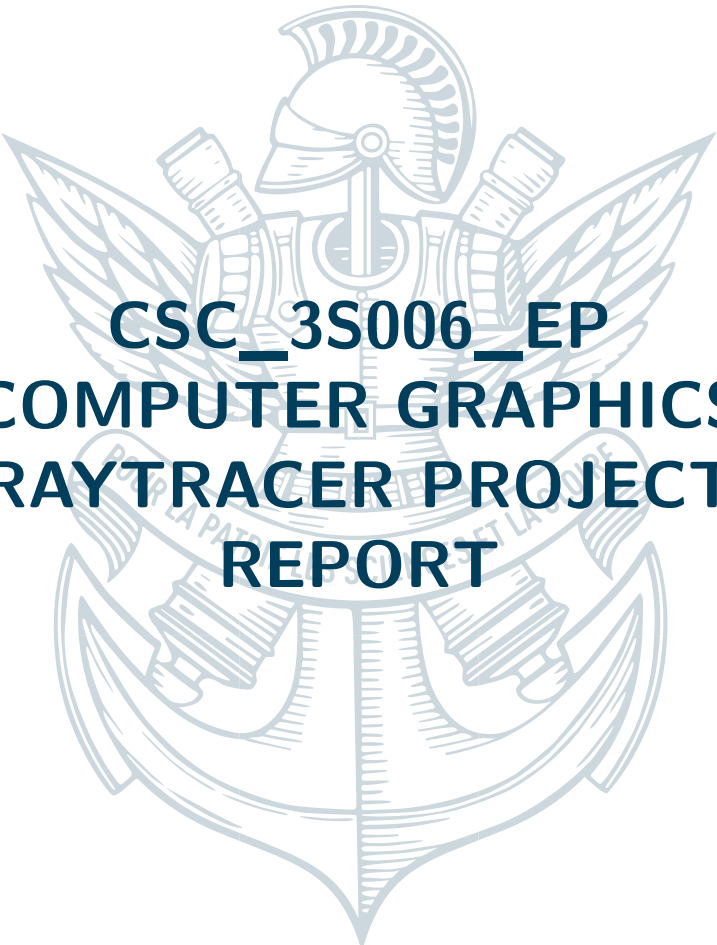




CSC_3S006_EP COMPUTER GRAPHICS RAYTRACER PROJECT REPORT

May 18th, 2026

Karl Khalaf



RAYTRACER

1. OVERVIEW

The goal of this project was to progressively implement a C++ raytracer/path tracer following Labs 1–4. The final renderer supports analytic spheres, diffuse Lambertian materials, mirror reflection, point-light direct illumination with hard shadows, recursive indirect illumination, Gaussian anti-aliasing, OBJ triangle meshes, and BVH acceleration.

The scene is a box-like room made from large spheres approximating planes. The final sphere render contains two balls: a light yellow opaque diffuse sphere and a mirror sphere. It is the result of playing with the code after the final submission for the purpose of this report. The mesh render, the one shown in the final repository submission, uses the provided `cat.obj`, scaled by 0.6 and translated by $(0, -10, 0)$, as in the lab notes.

Parameter	Value	Parameter	Value
Resolution	512×512	Camera center	$(0, 0, 55)$
Horizontal FOV	60°	Point light	$(-10, 20, 40)$
Light intensity	10^7	Gamma	2.2
Max. bounces	5	Final sampling	128 paths/pixel
Sphere albedo	$(1, 0.95, 0.6)$	Cat mesh	3954 triangles

Table 1: Main parameters used in the submitted renders.

2. LAB 1: CAMERA RAYS, SPHERE INTERSECTIONS, DIRECT LIGHTING, SHADOWS, AND MIRRORS

For each pixel (i, j) , a primary ray is shot from the camera center Q through the virtual image plane. With image size $W \times H$ and field of view α , the focal distance is

$$f = \frac{W}{2 \tan(\alpha/2)},$$

so the direction before normalization is approximately

$$\mathbf{u}_{ij} = \left(j + \frac{1}{2} - \frac{W}{2}, \frac{H}{2} - i - \frac{1}{2}, -f \right).$$

The sphere intersection code solves the quadratic equation obtained by substituting the ray $X(t) = O + t\mathbf{u}$ into the sphere equation $\|X - C\|^2 = R^2$:

$$t^2 + 2t\langle \mathbf{u}, O - C \rangle + \|O - C\|^2 - R^2 = 0.$$

Using the reduced discriminant

$$\Delta = \langle \mathbf{u}, O - C \rangle^2 - (\|O - C\|^2 - R^2),$$

the two possible roots are

$$t_1 = \langle \mathbf{u}, C - O \rangle - \sqrt{\Delta}, \quad t_2 = \langle \mathbf{u}, C - O \rangle + \sqrt{\Delta}.$$

The implementation keeps the nearest non-negative root and returns the hit point $P = O + t\mathbf{u}$ and normal $N = (P - C)/R$. The scene intersection routine then loops through all objects and keeps the smallest positive t , which is essential for correct visibility and shadow testing.

For diffuse direct illumination, I used the Lambertian BRDF ρ/π . A shadow ray is sent from $P + \varepsilon N$ toward the point light S to avoid self-intersection artifacts. If the light is visible, the direct contribution is

$$L_{\text{direct}} = \frac{I}{4\pi d^2} V_P(S) \max(0, N \cdot \omega_l) \frac{\rho}{\pi}, \quad \omega_l = \frac{S - P}{\|S - P\|}, \quad d = \|S - P\|.$$

Mirror surfaces are handled recursively with the perfect specular reflection direction

$$\mathbf{r} = \mathbf{u} - 2\langle \mathbf{u}, \mathbf{N} \rangle \mathbf{N}.$$

In the code, a mirror hit immediately launches `getColor` on the reflected ray:

```
if (objects[object_id]->mirror) {
    Ray reflRay(P + 1e-5 * N, ray.u - 2*dot(ray.u, N) * N);
    return getColor(reflRay, recursion_depth + 1);
}
```

3. LAB 2: INDIRECT LIGHTING AND ANTI-ALIASING

Indirect lighting evaluates the diffuse rendering equation stochastically. For a non-emissive diffuse surface, the outgoing radiance can be written as

$$L_o(x, \omega_o) = \frac{\rho}{\pi} \int_{\Omega} L_i(x, \omega_i) \langle \omega_i, \mathbf{N} \rangle d\omega_i.$$

Instead of integrating over the hemisphere deterministically, the code samples one cosine-weighted direction per bounce. With $r_1, r_2 \sim U(0, 1)$, the local sample is

$$x = \cos(2\pi r_1) \sqrt{1 - r_2}, \quad y = \sin(2\pi r_1) \sqrt{1 - r_2}, \quad z = \sqrt{r_2},$$

whose probability density is $p(\omega) = \cos \theta / \pi$. The local z axis is then aligned with the surface normal using two tangent vectors T_1, T_2 . This cancels the Lambertian cosine/pdf factors, so the indirect estimate becomes a recursive color multiplied componentwise by the albedo:

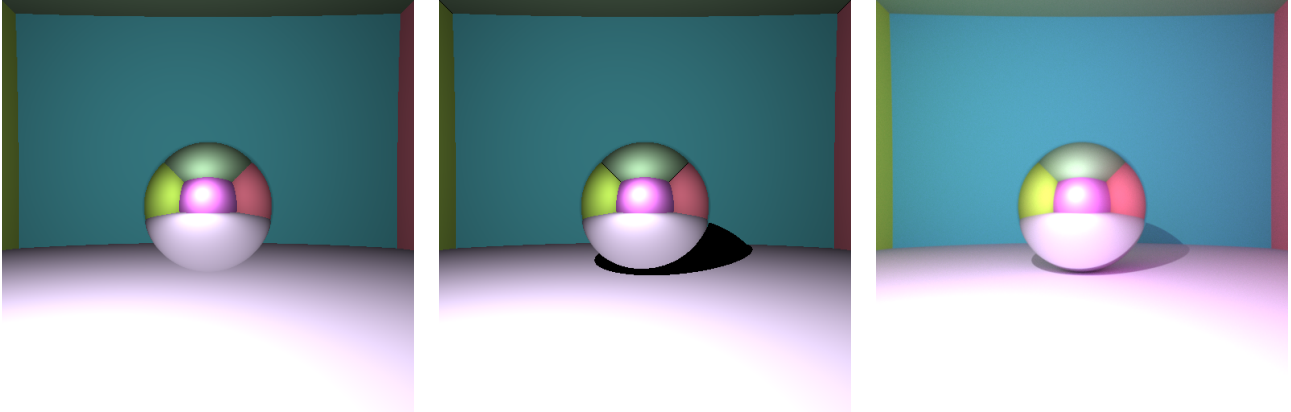
```
double x = cos(2.0*M_PI*r1) * sqrt(1.0-r2);
double y = sin(2.0*M_PI*r1) * sqrt(1.0-r2);
double z = sqrt(r2);
Vector wi = x*T1 + y*T2 + z*N;
wi.normalize();
Vector Li = getColor(Ray(P + 1e-5*N, wi), recursion_depth + 1);
indirect = mult(objects[object_id]->albedo, Li);
```

The recursion is truncated after 5 bounces, as required, and I did not use Russian roulette.

Anti-aliasing is implemented by integrating the radiance over a small neighborhood of each pixel rather than always sampling exactly at the pixel center. For pixel area A_{ij} , the target is

$$L_{ij} = \int_{A_{ij}} L_i(x, \omega_i(x)) h_{ij}(x) dx,$$

where I used a Gaussian filter h_{ij} centered at the pixel. In practice, each pixel averages 128 camera rays with Box-Muller jitter (g_x, g_y) added to the pixel center. The final average is gamma corrected only at the end using $c_{\text{out}} = 255(c_{\text{linear}}/255)^{1/\gamma}$ and clamped to $[0, 255]$.



(a) Lab 1: diffuse/mirror.

(b) Direct shadows.

(c) Indirect lighting.

Figure 1: Progression from basic sphere raytracing to shadows and path-traced indirect lighting.

4. LABS 3–4: MESHES, MOLLER–TRUMBORE, BOUNDING BOXES, AND BVH

The OBJ loader stores vertices, normals, UVs, and triangle indices. Polygonal faces with more than three vertices are converted to triangles by fan triangulation, so the cat model becomes 3954 triangles.

For each triangle (A, B, C) , I used the Moller–Trumbore ray-triangle test. Writing $e_1 = B - A$ and $e_2 = C - A$, the ray hit satisfies

$$O + t\mathbf{u} = A + \beta e_1 + \gamma e_2,$$

with acceptance conditions

$$\beta \geq 0, \quad \gamma \geq 0, \quad \beta + \gamma \leq 1, \quad t > \varepsilon.$$

When vertex normals are available, the final normal is interpolated as

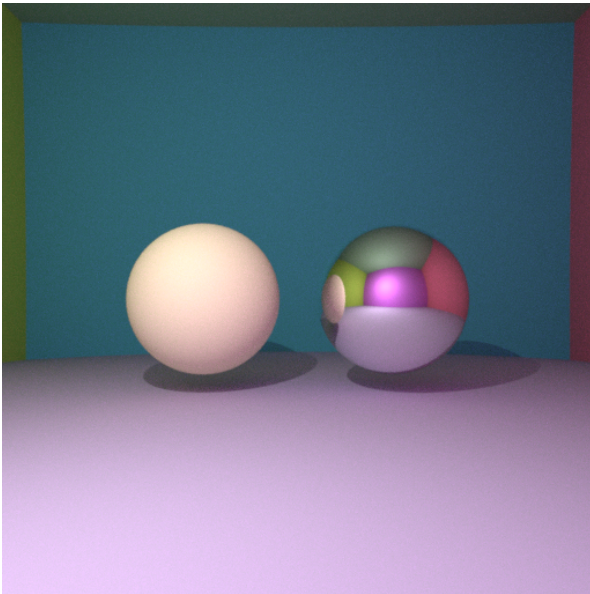
$$N = (1 - \beta - \gamma)N_A + \beta N_B + \gamma N_C,$$

then normalized and flipped if it points in the same direction as the incoming ray. The core of the code is:

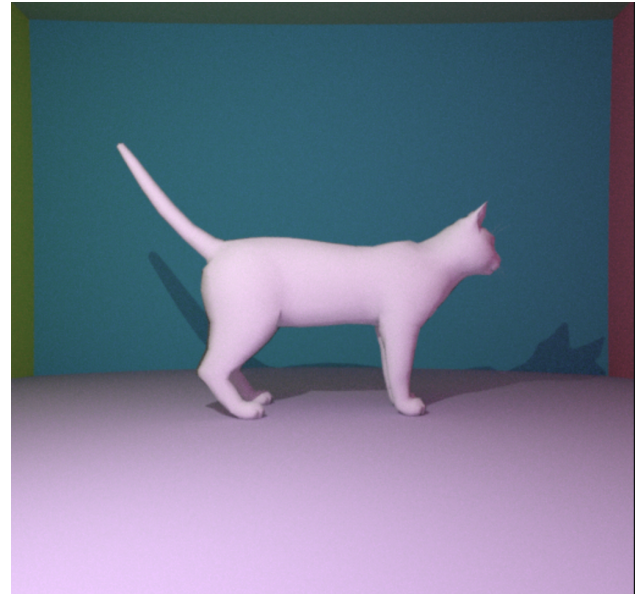
```
Vector pvec = cross(ray.u, e2);
double det = dot(e1, pvec);
if (fabs(det) < 1e-8) return false;
double invDet = 1.0 / det;
double beta = dot(ray.O - A, pvec) * invDet;
Vector qvec = cross(ray.O - A, e1);
double gamma = dot(ray.u, qvec) * invDet;
double cur_t = dot(e2, qvec) * invDet;
```

The first acceleration step is an axis-aligned bounding box test. For each coordinate axis, the ray is intersected with the two planes of the box, producing an interval of valid t values. The ray hits the box if the three intervals overlap. The implementation updates $t_{\min}$ and $t_{\max}$ for all axes and rejects the box when $t_{\min} > t_{\max}$.

Lab 4 extends this idea into a BVH. Each node stores $b_{\min}$, $b_{\max}$, a triangle range, and child indices. The tree is built recursively: compute the node bounding box, choose the largest centroid-spread axis, sort triangle indices on that axis, split in the middle, and stop at leaves with at most four triangles. Traversal first rejects rays that miss the node box, then either tests the few triangles in a leaf or recursively visits the two children, visiting the nearer child first when possible. This turns mesh intersection from an almost linear scan over all triangles into a logarithmic-style hierarchical search.



(a) Final two-sphere scene.



(b) Cat mesh render with BVH.

Figure 2: Final validation renders: analytic primitives and triangle mesh geometry.

5. PERFORMANCE AFTER BVH

Using time `./raytracer`, we compute the rendering times for the cat with 32 paths per pixel, as well as 128 paths per pixel.

Paths per pixel	Rendering time with BVH
32	2.206 s
128	8.163 s

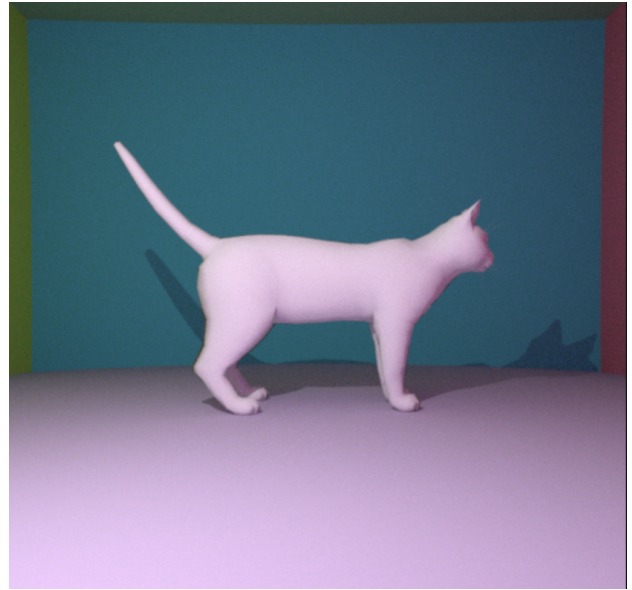
Table 2: Rendering time after BVH acceleration.

6. RESULTS, PERFORMANCE, AND CONCLUSION

Figure 3 compares two cat renders with different sampling counts. The geometry is identical, but the 32 paths/pixel version has stronger Monte Carlo noise in the shadows and on the walls. Increasing to 128 paths/pixel reduces this noise and gives smoother indirect illumination.



(a) 32 paths per pixel, high light intensity.



(b) 128 paths per pixel, low light intensity.

Figure 3: Effect of increasing the number of Monte Carlo paths per pixel.

The final sphere image was rendered at 512×512 using 128 paths/pixel and 5 bounces. The code uses OpenMP with `schedule(dynamic, 1)` on the outer image loop and one random generator per thread, which matches the lecture-note warning that the standard random engine should not be shared unsafely across OpenMP threads. Exact timing depends on the machine and should be measured with `time ./raytracer`; however, the main performance improvement for the mesh case comes from BVH traversal, because camera rays, shadow rays, and indirect rays all repeatedly call the intersection routine.

Overall, the submitted raytracer implements the required project features: diffuse and mirror surfaces, direct lighting and hard shadows, indirect lighting for point light scenes without Russian roulette, anti-aliasing, ray-mesh intersection, and BVH acceleration. The final images show both the visual progression of the labs and the ability to combine analytic primitives with triangle meshes in the same recursive path-tracing framework.